Teoría de Algoritmos
(TB024/75.29) Curso Buchwald - Genender

Trabajo Práctico 1
Algoritmos Greedy en la Nación del Fuego

3 de octubre de 2025

Facundo Camilo Stifman
112541

Juan Ignacio Moore
112479

Avril Victoria Morfeo Zerbi
112563

Martin Ariel Kupa
112560

Índice

1. Análisis algorítmico **3**
1.1. Enunciado del problema . . . 3
1.2. Algoritmos estudiados . . . 3
1.3. Resultados empíricos . . . 4
1.3.1. Por duración . . . 4
1.3.2. Por importancia . . . 4
1.3.3. Por coeficiente . . . 5
1.4. Conclusión . . . 5

2. Demostración **6**
2.1. Orden de soluciones óptimas . . . 6
2.2. Equivalencia de soluciones ordenadas . . . 7
2.3. Conclusión . . . 7

3. Mediciones **7**
3.1. Conjunto balanceado . . . 8
3.1.1. Error del ajuste . . . 8
3.2. Conjunto mayor duración . . . 9
3.2.1. Error del ajuste . . . 9
3.3. Conjunto mayor importancia . . . 10
3.3.1. Error del ajuste . . . 10
3.4. Conclusión . . . 10

A. Resultados de las mediciones **12**

B. Código de generación de datos **13**

C. Anexo **15**
C.1. Greedy . . . 15
C.2. Orden Algoritmo . . . 15

1. Análisis algorítmico

1.1. Enunciado del problema

Dado un conjunto W de n pares (t_i, b_i):

$$W = \left\{(t_i, b_i) \in \mathbb{R}^2_{>0} | i \in [n]\right\}$$

Donde cada par representa una batalla, siendo t_i su duración y b_i su importancia.

Sea un orden cualquiera de las batallas $X = (w_1, \ldots, w_n)$. Definimos F_i como el tiempo de finalización de la batalla w_i según dicho orden, es decir, la suma de las duraciones de todas las batallas que la preceden más la suya propia:

$$F_i = \sum_{k=1}^{i} t_k.$$

Se busca una secuencia S de batallas w que minimice la suma ponderada $f(X)$ de los tiempos de finalización de cada batalla:

$$f(X) := \sum_{i=1}^{n} b_i \cdot F_i$$
$$S \in \arg\min_X f(X)$$

1.2. Algoritmos estudiados

A fin de minimizar $f(S)$ se proponen tres algoritmos greedy que construyen S elemento a elemento, pero en base a criterios distintos. Dichos algoritmos comparten la misma estructura lógica:

1. Se consideran las batallas aún no realizadas, que constituyen un estado local.
2. Se toma una batalla en particular en base a un criterio predefinido, de manera que se obtenga un óptimo local.
3. Se agrega la batalla elegida como último elemento de la secuencia S, modificando el estado actual al marcar la batalla como realizada.
4. Se repite desde el paso uno hasta agotar todas las batallas.

Los criterios de selección de cada algoritmo, que a su vez se utilizarán como sus respectivos nombres, son:

- Por duración: En cada iteración se elige la batalla de menor duración, de manera que los tiempos de finalización de las batallas subsecuentes sean menores.
- Por importancia: En cada iteración se elige la batalla de mayor importancia, de forma que la finalización temprana contrarreste el aporte de la importancia.
- Por coeficiente: En cada iteración se elige la batalla de menor cociente duración/importancia. Definimos a este cociente como el *coeficiente de coste* de la batalla i:

  $$c_i := \frac{t_i}{b_i}.$$

  Este valor combina los criterios anteriores y determina la relevancia relativa de cada batalla.

Los algoritmos propuestos se pueden implementar de forma sencilla como un *sort* en base al criterio.

```python
from src.batalla import Batalla

def por_menor_coeficiente(batallas: list[Batalla]) -> list[Batalla]:
    batallas_ordenadas = sorted(batallas, key=lambda b: b.duracion/b.importancia)
    return batallas_ordenadas

def por_menor_duracion(batallas: list[Batalla]) -> list[Batalla]:
    batallas_ordenadas = sorted(batallas, key=lambda b: b.duracion)
    return batallas_ordenadas

def por_mayor_importancia(batallas: list[Batalla]) -> list[Batalla]:
    batallas_ordenadas = sorted(batallas, key=lambda b: b.importancia, reverse=True)
    return batallas_ordenadas
```

En CPython, la implementación utilizada para la ejecución de estos algoritmos, la complejidad temporal de *sorted* es $O(n \log n)$ (TimeComplexity 2023).

1.3. Resultados empíricos

Procedemos a comparar los resultados de cada algoritmo sobre los conjuntos de datos proveídos por la cátedra, para los cuales se conoce la solución óptima.

1.3.1. Por duración

El algoritmo por duración logra una solución óptima en dos conjuntos, los más pequeños, sin embargo falla para los cinco restantes. Esta falla se genera porque, al ordenar las batallas solo por tiempo de duración, se minimiza la suma de tiempos de finalización, sin tener en cuenta el peso que conlleva cada una. Esto implica que una batalla corta pero poco importante tiene más relevancia sobre una más larga pero con más importancia. En consecuencia, si una batalla con b_i muy grande se posterga, su F_i aumenta, y al multiplicarla por b_i el valor final incrementa significativamente, empeorando la suma ponderada total.

```
    datos/10.txt
    Coeficiente de impacto: 309600.0
    OPTIMO. Esperado: 309600.0, Obtenido: 309600.0
    datos/50.txt
    Coeficiente de impacto: 5218700.0
    OPTIMO. Esperado: 5218700.0, Obtenido: 5218700.0
    datos/100.txt
    Coeficiente de impacto: 904678219.0
    SUBOPTIMO. Esperado: 780025365.0, Obtenido: 904678219.0
    datos/1000.txt
    Coeficiente de impacto: 91611493634.0
    SUBOPTIMO. Esperado: 74329021942.0, Obtenido: 91611493634.0
    datos/5000.txt
    Coeficiente de impacto: 2261665165125.0
    SUBOPTIMO. Esperado: 1830026958236.0, Obtenido: 2261665165125.0
    datos/10000.txt
    Coeficiente de impacto: 8926877144801.0
    SUBOPTIMO. Esperado: 7245315862869.0, Obtenido: 8926877144801.0
    datos/100000.txt
    Coeficiente de impacto: 896816688147043.0
    SUBOPTIMO. Esperado: 728684685661017.0, Obtenido: 896816688147043.
```

1.3.2. Por importancia

El algoritmo por importancia no logra encontrar una solución óptima para ninguno de los conjuntos de datos de la cátedra. En este caso ocurre algo similar a ordenar las batallas por duración, pero en sentido contrario. Si una batalla con b_i muy grande pero de larga duración

se ubica al inicio, aumenta considerablemente los tiempos de finalización de todas las batallas posteriores, afectando negativamente la suma ponderada final.

```
datos/10.txt
Coeficiente de impacto: 352800.0
SUBOPTIMO. Esperado: 309600.0, Obtenido: 352800.0
datos/50.txt
Coeficiente de impacto: 7584000.0
SUBOPTIMO. Esperado: 5218700.0, Obtenido: 7584000.0
datos/100.txt
Coeficiente de impacto: 1010681953.0
SUBOPTIMO. Esperado: 780025365.0, Obtenido: 1010681953.0
datos/1000.txt
Coeficiente de impacto: 94697143232.0
SUBOPTIMO. Esperado: 74329021942.0, Obtenido: 94697143232.0
datos/5000.txt
Coeficiente de impacto: 2318576356487.0
SUBOPTIMO. Esperado: 1830026958236.0, Obtenido: 2318576356487.0
datos/10000.txt
Coeficiente de impacto: 9226806713585.0
SUBOPTIMO. Esperado: 7245315862869.0, Obtenido: 9226806713585.0
datos/100000.txt
Coeficiente de impacto: 927525303397289.0
SUBOPTIMO. Esperado: 728684685661017.0, Obtenido: 927525303397289.0
```

1.3.3. Por coeficiente

El algoritmo por coeficiente es el único en obtener una solución óptima en todos los conjuntos de datos. Eso se debe a que el método óptimo logra un balance entre los tiempos de finalización F_i con los pesos b_i de cada batalla, considerando no solo cuánto tarda una batalla sino también cuán importante es, mediante el cociente $\frac{t_i}{b_i}$, que determina la relevancia relativa y, en consecuencia, el orden en que deben llevarse a cabo.

```
datos/10.txt
Coeficiente de impacto: 309600.0
OPTIMO. Esperado: 309600.0, Obtenido: 309600.0
datos/50.txt
Coeficiente de impacto: 5218700.0
OPTIMO. Esperado: 5218700.0, Obtenido: 5218700.0
datos/100.txt
Coeficiente de impacto: 780025365.0
OPTIMO. Esperado: 780025365.0, Obtenido: 780025365.0
datos/1000.txt
Coeficiente de impacto: 74329021942.0
OPTIMO. Esperado: 74329021942.0, Obtenido: 74329021942.0
datos/5000.txt
Coeficiente de impacto: 1830026958236.0
OPTIMO. Esperado: 1830026958236.0, Obtenido: 1830026958236.0
datos/10000.txt
Coeficiente de impacto: 7245315862869.0
OPTIMO. Esperado: 7245315862869.0, Obtenido: 7245315862869.0
datos/100000.txt
Coeficiente de impacto: 728684685661017.0
OPTIMO. Esperado: 728684685661017.0, Obtenido: 728684685661017.0
```

1.4. Conclusión

De los algoritmos estudiados, el único con la posibilidad de generar una solución óptima para cualquier conjunto es el algoritmo por coeficientes. Los algoritmos por duración e importancia producen aproximaciones subóptimas para ciertos conjuntos. De estos últimos, el algoritmo por importancia produce resultados notablemente inferiores a los del algoritmo por duración.

2. Demostración

Procedemos a demostrar la optimalidad del algoritmo por coeficientes mediante inversiones. Para esto planteamos dos afirmaciones que una vez probadas, nos permitirán concluir el resultado.

2.1. Orden de soluciones óptimas

Procedemos a demostrar la primera afirmación: toda secuencia óptima debe estar ordenada por coeficientes, lo cual probaremos mediante el absurdo.

Sea $X^* = (w_1, \ldots, w_n)$ una secuencia óptima de batallas, donde cada batalla $w_i = (t_i, b_i)$ está formada por su duración t_i y su importancia b_i.

Definimos el coeficiente asociado a cada batalla como:

$$c_i := \frac{t_i}{b_i}.$$

Supongamos ahora que existen dos batallas consecutivas w_i, w_j en X^* (tomando $j = i + 1$) tales que sus coeficientes están invertidos, es decir, $c_i > c_j$. A continuación procedemos a deshacer dicha inversión y estudiar su efecto en la función a optimizar.

Considere los términos de f correspondientes a la primera inversión de X^*:

$$f(X^*) = \left(\sum_{k=1}^{i} F_k b_k\right) + F_i b_i + F_j b_j + \left(\sum_{k=j+1}^{n} F_k b_k\right)$$

Si t_c es la sumatoria de duraciones anteriores a la i-ésima batalla:

$$t_c := \sum_{k=1}^{i-1} t_k$$

Entonces los tiempos de finalización pueden expresarse como:

$$F_i = t_c + t_i, \qquad F_j = t_c + t_i + t_j.$$

Podemos reescribir:

$$f(X^*) = \left(\sum_{k=1}^{i} F_k b_k\right) + (t_c + t_i) b_i + (t_c + t_i + t_j) b_j + \left(\sum_{k=j+1}^{n} F_k b_k\right)$$

Al deshacer la inversión obtenemos otra secuencia $X' = (w_1, \ldots, w_j, w_i, \ldots, w_n)$ que difiere únicamente en la posición relativa de las batallas w_i y w_j (cuyos coeficientes son c_i y c_j):

$$f(X') = \left(\sum_{k=1}^{i} F_k b_k\right) + (t_c + t_j) b_j + (t_c + t_j + t_i) b_i + \left(\sum_{k=j+1}^{n} F_k b_k\right)$$

Definimos:

$$C_1 = \left(\sum_{k=1}^{i} F_k b_k\right), \quad C_2 = \left(\sum_{k=j+1}^{n} F_k b_k\right)$$

Donde C_1 representa el aporte acumulado de las batallas anteriores a i, y C_2 el aporte acumulado de las batallas posteriores a j. De esta manera, el análisis puede concentrarse únicamente en las posiciones i y j.

Para analizar cuál de las dos secuencias produce un menor valor en la función que buscamos minimizar, consideramos la diferencia entre sus costos:

$$\begin{aligned}
f(X') - f(X^*) &= \Big(C_1 + (t_c + t_j)b_j + (t_c + t_j + t_i)b_i + C_2\Big) - \\
&\quad \Big(C_1 + (t_c + t_i)b_i + (t_c + t_i + t_j)b_j + C_2\Big) \\
&= \cancel{C_1 - C_1} + \cancel{t_c b_j - t_c b_j} + \cancel{t_j b_j - t_j b_j} + \cancel{t_c b_i - t_c b_i} + \\
&\quad \cancel{t_i b_i - t_i b_i} + t_j b_i - t_i b_j + \cancel{C_2 - C_2} \\
&= t_j b_i - t_i b_j
\end{aligned}$$

Observamos que la diferencia $f(X') - f(X^*)$ depende únicamente de los coeficientes de las batallas i y j. Si analizamos el signo de esta expresión bajo la hipótesis de que $c_i > c_j$:

$$\begin{aligned}
\frac{t_i}{b_i} &> \frac{t_j}{b_j} \\
t_i b_j &> t_j b_i \\
t_j b_i - t_i b_j &< 0 \\
f(X') - f(X^*) &< 0 \\
f(X') &< f(X^*)
\end{aligned}$$

Esto implica que al deshacer la inversión obtuvimos una solución mejor que X^*. Dicho absurdo surge de suponer que la solución óptima tiene inversiones en primer lugar. Por lo tanto, ninguna solución óptima admite inversiones respecto de los coeficientes de sus batallas, pues de lo contrario no sería óptima.

2.2. Equivalencia de soluciones ordenadas

Ahora se prueba la segunda afirmación. Suponiendo la existencia de una solución óptima ordenada $O = (w_1, \ldots, w_n)$ tal que sus coeficientes cumplen $c_i \leq c_j \ \forall i < j$. Toda secuencia ordenada de las batallas puede diferir solo en el orden relativo de coeficientes equivalentes. Partiendo de que $c_i = c_j$ y por el mismo razonamiento previo podemos concluir que, de permutar dichos elementos en otra secuencia X', dicha secuencia sigue estando ordenada y se cumple que $f(X^*) - f(X') = 0$. Por lo tanto, toda secuencia ordenada es igual de óptima.

2.3. Conclusión

Hemos probado que:

1. Ninguna secuencia óptima puede contener inversiones, es decir, toda secuencia óptima está ordenada por coeficientes.
2. Cualquier secuencia ordenada por coeficiente es igualmente óptima.

De esta forma, concluimos que el algoritmo greedy por coeficiente siempre genera una solución óptima para cualquier instancia del problema. □

3. Mediciones

Para corroborar la complejidad temporal documentada se generaron conjuntos de datos de tamaños y rangos diversos, los cuales se utilizaron para tomar mediciones del tiempo de ejecución de los algoritmos propuestos. En concreto, los conjuntos generados son:

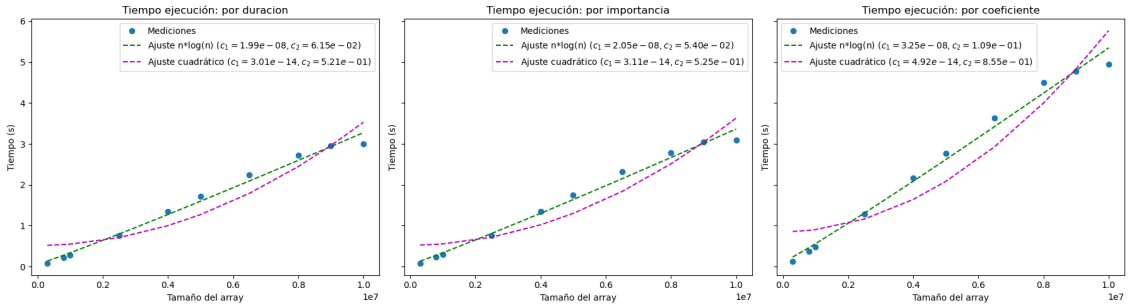
- Balanceado: Compuesto por batallas con $b, d \in [1, 100]$
- Mayor duración: Compuesto por batallas con $d \in [1, 1000], b \in [1, 10]$
- Mayor importancia: Compuesto por batallas con $b \in [1, 1000], d \in [1, 10]$

Los valores de duración e importancia de los conjuntos mencionados fueron generados de manera aleatoria mediante una distribución uniforme. Para todos los conjuntos se generaron múltiples listas de batallas con largos de entre 300,000 y 10,000,000. Dichas listas se utilizaron como entrada para cada uno de los algoritmos propuestos, realizando diez pruebas y tomando la media aritmética de los tiempos de ejecución (véase el apéndice A). Luego, se ajustaron las siguientes curvas a los datos obtenidos mediante el método de cuadrados mínimos:

$$f_1(x) = c_1 x \log(x) + c_2 \quad f_2(x) = c_1 x^2 + c_2$$

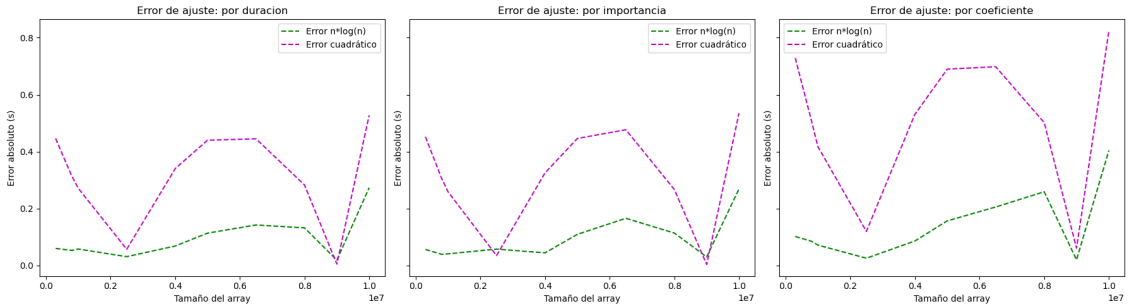
Finalmente, se recopilaron los resultados y se generaron gráficos que muestran la relación entre el tamaño de entrada y el tiempo de ejecución, así como el error de ajuste respecto a las distintas curvas. A continuación se presentan los resultados obtenidos para cada conjunto.

3.1. Conjunto balanceado



3.1.1. Error del ajuste

Error absoluto:

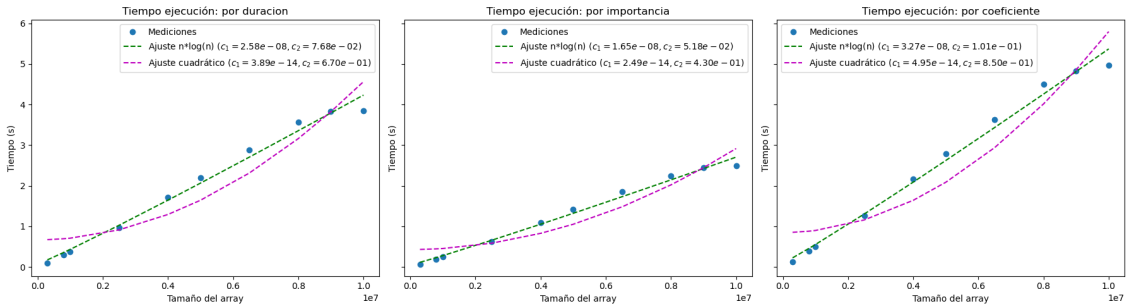


Error cuadrático total:

```
Criterio duracion
Para n*log(n): 0.1405130481831952
Para n^2: 1.2396349686242265

Criterio importancia
Para n*log(n): 0.1368589133382353
Para n^2: 1.2556144732112386

```

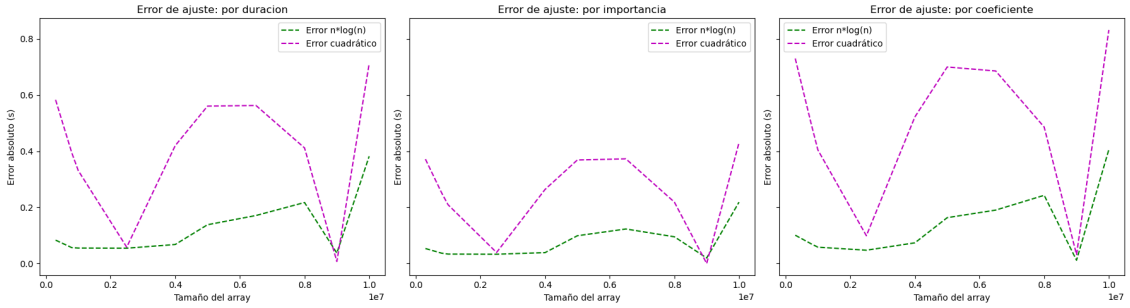
```
Criterio coeficiente
Para n*log(n): 0.3273336912261867
Para n^2: 3.1574973470269416
```

3.2. Conjunto mayor duración



3.2.1. Error del ajuste

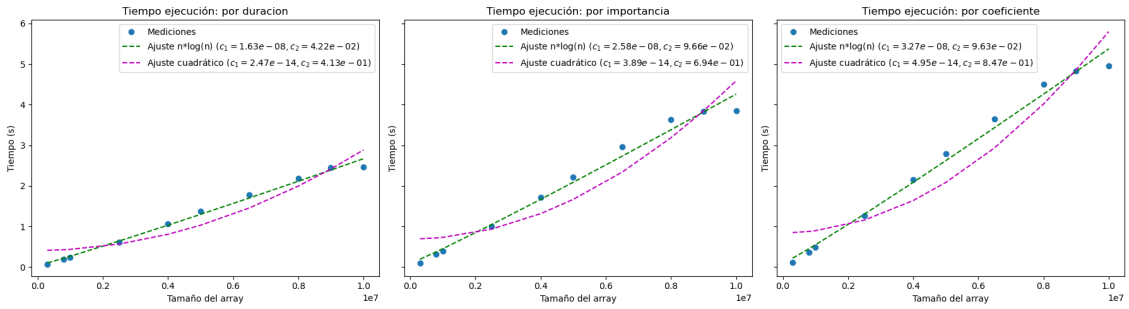
Error absoluto:



Error cuadrático total:

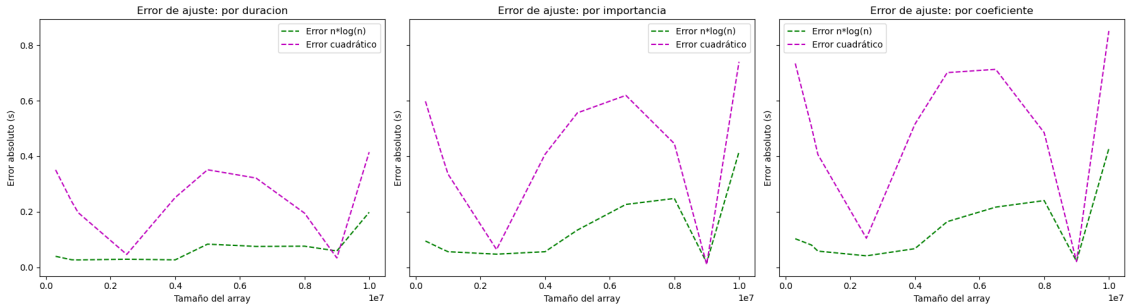
```
Criterio duracion
Para n*log(n): 0.26385086378859784
Para n^2: 2.0922653536242706

Criterio importancia
Para n*log(n): 0.09026633945341109
Para n^2: 0.8249674114366081

Criterio coeficiente
Para n*log(n): 0.31296449180417774
Para n^2: 3.112003217096628
```

3.3. Conjunto mayor importancia



3.3.1. Error del ajuste

Error absoluto:



Error cuadrático total:

```
Criterio duracion
Para n*log(n): 0.06542035135976598
Para n^2: 0.7203445500299227

Criterio importancia
Para n*log(n): 0.32558359309981155
Para n^2: 2.244643574269059

Criterio coeficiente
Para n*log(n): 0.3411352496260656
Para n^2: 3.1969562277170898
```

3.4. Conclusión

Se observa que, para todos los casos evaluados, el tiempo de ejecución crece de forma semejante. De las curvas estudiadas la función linearítmica resulta en el menor error cuadrático, lo cual es consistente con la complejidad teórica esperada.

Cabe notar que el algoritmo por coeficientes resulta en el mayor tiempo de ejecución para los conjuntos estudiados, una posible razón es el coste del cálculo de los coeficientes. A su vez, los algoritmos por importancia y por duración tardan considerablemente menos cuando el rango de valores a ordenar está más acotado. Por ejemplo el algoritmo por importancia tarda aproximadamente 2,49 segundos en ordenar una lista de 10,000,000 de batallas del conjunto de mayor duración, mientras que para una lista del mismo largo del conjunto balanceado tarda aproximadamente 3,10 segundos. La diferencia representa una mejora del 19,68 %, en nuestro caso dicha mejora se puede atribuir al método de ordenamiento utilizado por *sorted*. CPython implementa *sorted* mediante el ordenamiento *Timsort* (PythonDocumentation 2025), el cual aprovecha cualquier orden ya presente en la

lista a ordenar. Al limitar el rango de valores posibles, aumenta la repetición y en consecuencia la probabilidad de sublistas ya ordenadas.

Referencias

PythonDocumentation (2025). *Sorting Techniques*. [Online; visitado el 9-Septiembre-2025]. URL: `https://docs.python.org/3/howto/sorting.html`.

TimeComplexity (2023). *TimeComplexity — The Python Wiki*. [Online; visitado el 31-Agosto-2025]. URL: `https://wiki.python.org/moin/TimeComplexity`.

A. Resultados de las mediciones

```
=== Conjunto tilt_b ===

Algoritmo por duracion
  Largo:   300000  Impacto total: 86587031309751.0  Tiempo promedio: 0.064554
  Largo:   800000  Impacto total: 617557675701968.0  Tiempo promedio: 0.192661
  Largo:  1000000  Impacto total: 963497475026337.0  Tiempo promedio: 0.241084
  Largo:  2500000  Impacto total: 6022380331955946.0  Tiempo promedio: 0.613670
  Largo:  4000000  Impacto total: 1.5419919943076396e+16  Tiempo promedio: 1.059845
  Largo:  5000000  Impacto total: 2.4078562657578984e+16  Tiempo promedio: 1.382448
  Largo:  6500000  Impacto total: 4.070455050446957e+16  Tiempo promedio: 1.779175
  Largo:  8000000  Impacto total: 6.167380244838551e+16  Tiempo promedio: 2.190647
  Largo:  9000000  Impacto total: 7.800633914176398e+16  Tiempo promedio: 2.449526
  Largo: 10000000  Impacto total: 9.633759500688683e+16  Tiempo promedio: 2.471255
Algoritmo por importancia
  Largo:   300000  Impacto total: 82579125302145.0  Tiempo promedio: 0.099687
  Largo:   800000  Impacto total: 588351225994701.0  Tiempo promedio: 0.311098
  Largo:  1000000  Impacto total: 918431341027725.0  Tiempo promedio: 0.397519
  Largo:  2500000  Impacto total: 5738689841429421.0  Tiempo promedio: 1.000925
  Largo:  4000000  Impacto total: 1.4694127674049484e+16  Tiempo promedio: 1.723334
  Largo:  5000000  Impacto total: 2.2958793840493904e+16  Tiempo promedio: 2.222783
  Largo:  6500000  Impacto total: 3.879547454166713e+16  Tiempo promedio: 2.956991
  Largo:  8000000  Impacto total: 5.877827909538346e+16  Tiempo promedio: 3.629291
  Largo:  9000000  Impacto total: 7.435105199160891e+16  Tiempo promedio: 3.835552
  Largo: 10000000  Impacto total: 9.179935810256094e+16  Tiempo promedio: 3.844472
Algoritmo por coeficiente
  Largo:   300000  Impacto total: 67140895667917.0  Tiempo promedio: 0.117537
  Largo:   800000  Impacto total: 478797311196310.0  Tiempo promedio: 0.372341
  Largo:  1000000  Impacto total: 747287809743770.0  Tiempo promedio: 0.490614
  Largo:  2500000  Impacto total: 4668701167828416.0  Tiempo promedio: 1.261413
  Largo:  4000000  Impacto total: 1.1955535133595468e+16  Tiempo promedio: 2.154657
  Largo:  5000000  Impacto total: 1.8673993752406824e+16  Tiempo promedio: 2.786368
  Largo:  6500000  Impacto total: 3.15581878723817e+16  Tiempo promedio: 3.652356
  Largo:  8000000  Impacto total: 4.78181847087074e+16  Tiempo promedio: 4.500956
  Largo:  9000000  Impacto total: 6.0489747952781576e+16  Tiempo promedio: 4.837492
  Largo: 10000000  Impacto total: 7.469096725184789e+16  Tiempo promedio: 4.947665

=== Conjunto balanceado ===

Algoritmo por duracion
  Largo:   300000  Impacto total: 76644437249012.0  Tiempo promedio: 0.077335
  Largo:   800000  Impacto total: 545972896819443.0  Tiempo promedio: 0.225949
  Largo:  1000000  Impacto total: 854596206163872.0  Tiempo promedio: 0.279590
  Largo:  2500000  Impacto total: 5337676015096177.0  Tiempo promedio: 0.765527
  Largo:  4000000  Impacto total: 1.3665314791920746e+16  Tiempo promedio: 1.342423
  Largo:  5000000  Impacto total: 2.1349380056730976e+16  Tiempo promedio: 1.713152
  Largo:  6500000  Impacto total: 3.6100774588072904e+16  Tiempo promedio: 2.237464
  Largo:  8000000  Impacto total: 5.4673705230161336e+16  Tiempo promedio: 2.729802
  Largo:  9000000  Impacto total: 6.921441786791603e+16  Tiempo promedio: 2.954586
  Largo: 10000000  Impacto total: 8.541142350537533e+16  Tiempo promedio: 3.003406
Algoritmo por importancia
  Largo:   300000  Impacto total: 76625103019685.0  Tiempo promedio: 0.075690
  Largo:   800000  Impacto total: 545972632984706.0  Tiempo promedio: 0.238667
  Largo:  1000000  Impacto total: 853902511148877.0  Tiempo promedio: 0.297324
  Largo:  2500000  Impacto total: 5343551525626577.0  Tiempo promedio: 0.753462
  Largo:  4000000  Impacto total: 1.3662899751015978e+16  Tiempo promedio: 1.347075
  Largo:  5000000  Impacto total: 2.1351916426292004e+16  Tiempo promedio: 1.747477
  Largo:  6500000  Impacto total: 3.6098262773667864e+16  Tiempo promedio: 2.314234
  Largo:  8000000  Impacto total: 5.466973184088931e+16  Tiempo promedio: 2.779924
  Largo:  9000000  Impacto total: 6.920989979523429e+16  Tiempo promedio: 3.044386
  Largo: 10000000  Impacto total: 8.54064543392165e+16  Tiempo promedio: 3.096298
Algoritmo por coeficiente
  Largo:   300000  Impacto total: 59908369058464.0  Tiempo promedio: 0.130887
  Largo:   800000  Impacto total: 426864400289191.0  Tiempo promedio: 0.378031
  Largo:  1000000  Impacto total: 667708015860956.0  Tiempo promedio: 0.487306
  Largo:  2500000  Impacto total: 4174487911269540.0  Tiempo promedio: 1.281999
  Largo:  4000000  Impacto total: 1.0679776465838488e+16  Tiempo promedio: 2.172591
  Largo:  5000000  Impacto total: 1.6690694887152528e+16  Tiempo promedio: 2.774061
  Largo:  6500000  Impacto total: 2.8216799567385976e+16  Tiempo promedio: 3.630772
```

```
  Largo:  8000000  Impacto total: 4.273666477596096e+16  Tiempo promedio: 4.503675
  Largo:  9000000  Impacto total: 5.41063384250186e+16  Tiempo promedio: 4.776813
  Largo: 10000000  Impacto total: 6.675411261643283e+16  Tiempo promedio: 4.947825

=== Conjunto tilt_t ===

Algoritmo por duracion
  Largo:   300000  Impacto total: 82512311501597.0  Tiempo promedio: 0.090952
  Largo:   800000  Impacto total: 587474636186879.0  Tiempo promedio: 0.300573
  Largo:  1000000  Impacto total: 917581424237775.0  Tiempo promedio: 0.377959
  Largo:  2500000  Impacto total: 5730967938829030.0  Tiempo promedio: 0.972125
  Largo:  4000000  Impacto total: 1.4698281604946972e+16  Tiempo promedio: 1.713133
  Largo:  5000000  Impacto total: 2.2977086467000816e+16  Tiempo promedio: 2.204480
  Largo:  6500000  Impacto total: 3.880295960453653e+16  Tiempo promedio: 2.878181
  Largo:  8000000  Impacto total: 5.878700501623103e+16  Tiempo promedio: 3.574190
  Largo:  9000000  Impacto total: 7.42950795089398e+16  Tiempo promedio: 3.831625
  Largo: 10000000  Impacto total: 9.174222204935259e+16  Tiempo promedio: 3.852343
Algoritmo por importancia
  Largo:   300000  Impacto total: 86612442641658.0  Tiempo promedio: 0.060496
  Largo:   800000  Impacto total: 616332858788114.0  Tiempo promedio: 0.193808
  Largo:  1000000  Impacto total: 963014553145756.0  Tiempo promedio: 0.245413
  Largo:  2500000  Impacto total: 6017431665763577.0  Tiempo promedio: 0.624844
  Largo:  4000000  Impacto total: 1.5426268393078756e+16  Tiempo promedio: 1.091828
  Largo:  5000000  Impacto total: 2.4106448703933052e+16  Tiempo promedio: 1.420600
  Largo:  6500000  Impacto total: 4.072498940017329e+16  Tiempo promedio: 1.853647
  Largo:  8000000  Impacto total: 6.168689889477354e+16  Tiempo promedio: 2.240654
  Largo:  9000000  Impacto total: 7.796783584933787e+16  Tiempo promedio: 2.444283
  Largo: 10000000  Impacto total: 9.629897071501147e+16  Tiempo promedio: 2.486771
Algoritmo por coeficiente
  Largo:   300000  Impacto total: 67185402517834.0  Tiempo promedio: 0.124038
  Largo:   800000  Impacto total: 477776473214901.0  Tiempo promedio: 0.386984
  Largo:  1000000  Impacto total: 746566917622318.0  Tiempo promedio: 0.494643
  Largo:  2500000  Impacto total: 4664094839505833.0  Tiempo promedio: 1.257965
  Largo:  4000000  Impacto total: 1.1959300025589752e+16  Tiempo promedio: 2.163314
  Largo:  5000000  Impacto total: 1.869540050109096e+16  Tiempo promedio: 2.786435
  Largo:  6500000  Impacto total: 3.1575429852976376e+16  Tiempo promedio: 3.625690
  Largo:  8000000  Impacto total: 4.783067656942934e+16  Tiempo promedio: 4.502018
  Largo:  9000000  Impacto total: 6.044377550937715e+16  Tiempo promedio: 4.825212
  Largo: 10000000  Impacto total: 7.465347425362618e+16  Tiempo promedio: 4.965646
```

B. Código de generación de datos

```python
import random
import sys
from src.batalla import Batalla

MODO_ESCRITURA = 'w'
ACLARACION = "Los datos a continuacion se encuentran en formato (F,b) donde 'F' es la suma de los tiempos de finalizacion correspondientes y 'b' la importacia de la batalla:\n"
GENERICO = "generico"
B_MAYOR_T = "b_mayor_t"
T_MAYOR_B = "t_mayor_b"

def generador_generico() -> Batalla:
  dur = random.randint(1, 100)
  imp = random.randint(1, 100)
  return Batalla(duracion=dur, importancia=imp)

def generar_batalla_b_mayor_t() -> Batalla:
  dur = random.randint(1, 10)
  imp = random.randint(1, 1000)
  return Batalla(duracion=dur, importancia=imp)

def generar_batalla_t_mayor_b() -> Batalla:
  dur = random.randint(1, 1000)
  imp = random.randint(1, 10)
```

```
  return Batalla(duracion=dur, importancia=imp)

def escribir_datos(cantidad_datos, ruta, callback_generador) -> None:
  with open(ruta, MODO_ESCRITURA) as archivo:
    archivo.write(ACLARACION)
    for _ in range(cantidad_datos):
      batalla = callback_generador()
      archivo.write(f'{batalla.duracion},{batalla.importancia}\n')

def generar(cantidad_datos, ruta_salida, generador):
  GENERADORES = {
      GENERICO: generador_generico,
      B_MAYOR_T: generar_batalla_b_mayor_t,
      T_MAYOR_B: generar_batalla_t_mayor_b,
  }

  if generador not in GENERADORES:
    print(f"Tipo de generador '{generador}' no reconocido. Opciones: {list(GENERADORES↩
        .keys())}")
    sys.exit(1)

  callback = GENERADORES.get(generador, generador_generico)
  escribir_datos(int(cantidad_datos), ruta_salida, callback)
```

C. Anexo

C.1. Greedy

Para el problema presentado se propusieron tres algoritmos greedy, cada uno basado en una regla sencilla que permite obtener dos óptimos locales y uno total. Para llegar a estas conclusiones, se reflexionó sobre el orden en que se recorren las batallas. En el primer algoritmo las batallas se ordenan por duración ascendente para minimizar los tiempos de finalización, lo que brinda un óptimo local al evitar que los tiempos de batallas subsecuentes acumulen la duración de batallas más largas; sin embargo, al existir un peso asociado a cada batalla, surge la necesidad de considerar una solución que contemple este aspecto. De manera similar, el algoritmo por importancia obtiene un óptimo local al priorizar las batallas más relevantes. Con esto se logra minimizar los productos de cada batalla al contrarrestar la importancia con una finalización temprana, pero no considera cómo este proceso afecta a los tiempos de finalización del resto de las batallas. Finalmente, para contemplar ambos factores, se presenta el algoritmo por coeficientes. Este último logra un balance entre ambos criterios al priorizar las batallas en base a un cociente entre su duración e importancia. Una batalla de gran importancia y larga duración no tendría excesiva prioridad ya que ambos factores se contrarrestan entre sí. Al priorizar aquellas batallas con menor coeficiente logramos optimizar en base a los criterios de los algoritmos anteriores de forma equitativa, lo que resulta en un óptimo total.

C.2. Orden Algoritmo

Si bien se demuestra que, entre los tres algoritmos presentados, ordenar mediante un coeficiente promedio de tiempo e importancia minimiza el impacto total, resulta relevante analizar el orden efectuado por cada algoritmo sobre los distintos arreglos de batallas. Es así, que al ejecutarlo sobre una muestra pequeña de treinta batallas, donde el tiempo y la importancia son valores dentro del rango de cero a cien, se obtienen los siguientes resultados:

```
Orden: [(t: 4.0, b: 22.0), (t: 18.0, b: 81.0), (t: 15.0, b: 64.0), (t: 26.0, b: 99.0),↩
     (t: 26.0, b: 83.0), (t: 32.0, b: 80.0), (t: 46.0, b: 94.0), (t: 38.0, b: 70.0), (↩
    t: 24.0, b: 41.0), (t: 34.0, b: 56.0), (t: 27.0, b: 37.0), (t: 56.0, b: 76.0), (t:↩
     47.0, b: 61.0), (t: 65.0, b: 80.0), (t: 38.0, b: 44.0), (t: 73.0, b: 78.0), (t: ↩
    59.0, b: 51.0), (t: 92.0, b: 72.0), (t: 21.0, b: 16.0), (t: 94.0, b: 59.0), (t: ↩
    27.0, b: 16.0), (t: 45.0, b: 25.0), (t: 86.0, b: 46.0), (t: 47.0, b: 23.0), (t: ↩
    97.0, b: 35.0), (t: 85.0, b: 26.0), (t: 98.0, b: 15.0), (t: 79.0, b: 10.0), (t: ↩
    68.0, b: 8.0), (t: 95.0, b: 9.0)]
Coeficiente de impacto: 627757.0
```

Al ejecutar el algoritmo por duración sobre la misma muestra:

```
Orden: [(t: 4.0, b: 22.0), (t: 15.0, b: 64.0), (t: 18.0, b: 81.0), (t: 21.0, b: 16.0),↩
     (t: 24.0, b: 41.0), (t: 26.0, b: 99.0), (t: 26.0, b: 83.0), (t: 27.0, b: 16.0), (↩
    t: 27.0, b: 37.0), (t: 32.0, b: 80.0), (t: 34.0, b: 56.0), (t: 38.0, b: 70.0), (t:↩
     38.0, b: 44.0), (t: 45.0, b: 25.0), (t: 46.0, b: 94.0), (t: 47.0, b: 61.0), (t: ↩
    47.0, b: 23.0), (t: 56.0, b: 76.0), (t: 59.0, b: 51.0), (t: 65.0, b: 80.0), (t: ↩
    68.0, b: 8.0), (t: 73.0, b: 78.0), (t: 79.0, b: 10.0), (t: 85.0, b: 26.0), (t: ↩
    86.0, b: 46.0), (t: 92.0, b: 72.0), (t: 94.0, b: 59.0), (t: 95.0, b: 9.0), (t: ↩
    97.0, b: 35.0), (t: 98.0, b: 15.0)]
Coeficiente de impacto: 730444.0
```

Finalmente, al aplicar el algoritmo por importancia:

```
Orden: [(t: 26.0, b: 99.0), (t: 46.0, b: 94.0), (t: 26.0, b: 83.0), (t: 18.0, b: 81.0)↩
    , (t: 32.0, b: 80.0), (t: 65.0, b: 80.0), (t: 73.0, b: 78.0), (t: 56.0, b: 76.0), ↩
    (t: 92.0, b: 72.0), (t: 38.0, b: 70.0), (t: 15.0, b: 64.0), (t: 47.0, b: 61.0), (t↩
    : 94.0, b: 59.0), (t: 34.0, b: 56.0), (t: 59.0, b: 51.0), (t: 86.0, b: 46.0), (t: ↩
    38.0, b: 44.0), (t: 24.0, b: 41.0), (t: 27.0, b: 37.0), (t: 97.0, b: 35.0), (t: ↩
    85.0, b: 26.0), (t: 45.0, b: 25.0), (t: 47.0, b: 23.0), (t: 4.0, b: 22.0), (t: ↩
    21.0, b: 16.0), (t: 27.0, b: 16.0), (t: 98.0, b: 15.0), (t: 79.0, b: 10.0), (t: ↩
    95.0, b: 9.0), (t: 68.0, b: 8.0)]
Coeficiente de impacto: 728060.0
```